Data-Structures & Algorithms

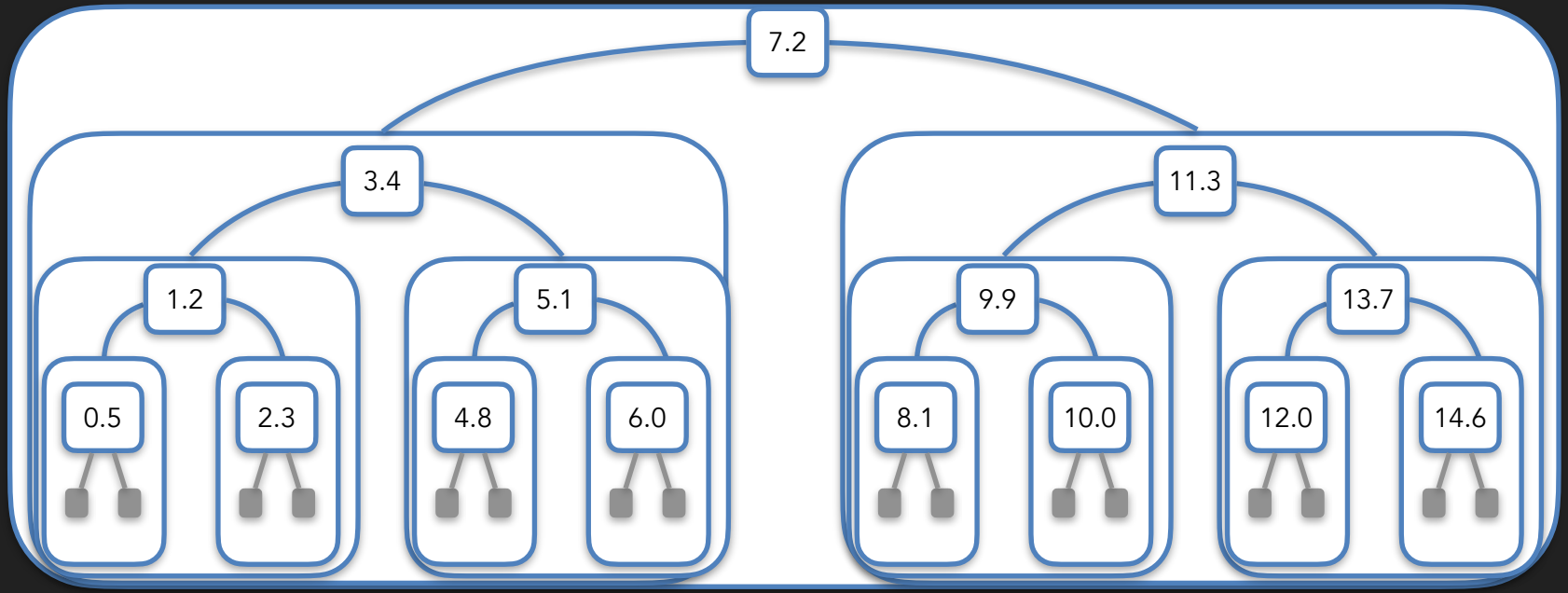
with

Manoj Prabhakaran

Lecture 29

Balanced BST

Recall: Binary Search Trees



A binary tree such that for any node with key K,

all nodes in its left subtree have keys < K, and

all nodes in its right subtree have keys ≥ K.

Recall: Binary Search Trees

- BSTs are a "workhorse" in data structures. E,g, in C++ STL:
  - `std::set`, `std::map`, `std::multiset` and `std::multimap`
  - Can hold any data type with `operator<`
- Recall: BST allows search, insertion, deletion of worst-case time O(h)
  - In particular, no need for Θ(n) time "rehashing" like in hash tables
- Also recall: $h = \Omega(\log n)$ (because $n = O(2^h)$)
- Goal: Maintain $h = \Theta(\log n)$
- In C++ STL (and in most other major implementations) this is done using a class of balanced binary trees called Red-Black Trees
  - There are other schemes that maintain slightly better balance and hence allow slightly faster lookup, but take (a const. factor) more time for insertion/deletion

Attendance Break!

Login using CSE LDAP id at

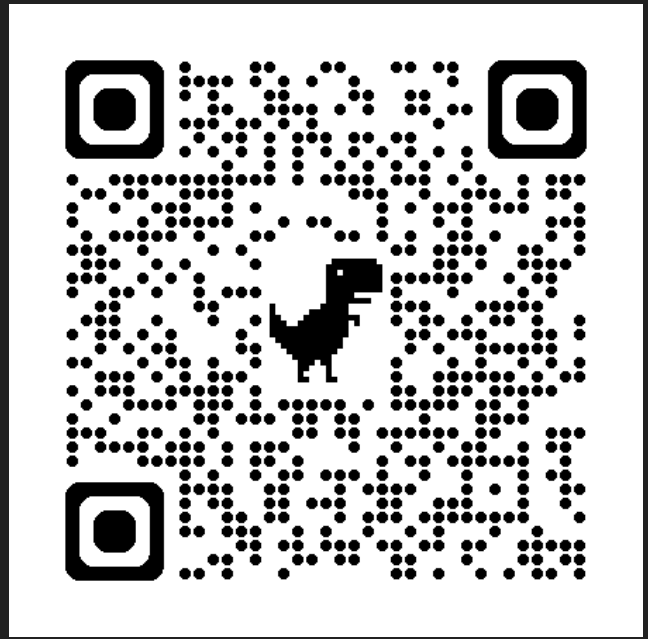
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Red-Black Trees

- Ideally, all root-to-leaf paths should be within ±1 of each other in length
  - E.g., in a nearly complete binary tree, in which case $h \leq \log_2 n$
- Maintaining a BST to be nearly complete is difficult/expensive
  - cf. Heap property can be maintained in O(log n) time per insertion/deletion
- We shall allow some slack:
  - All root-to-leaf paths are within a *factor 2* of each other in length
- Enough to guarantee that $h = O(\log n)$
  - If some leaf has depth h, every other leaf has depth ≥ h/2. So till depth h/2 a complete binary tree with $2^{h/2} - 1$ internal nodes. Overall, $n \geq 2^{h/2}$ and hence $\log n \geq h/2$ or $h \leq 2 \log n$

Red-Black Trees

- In a Red-Black tree, every node is coloured black or red
- Main invariants:
  - Every red node has a black parent
  - Every node has a unique *black-height*
    - The number of black nodes in the path from that node to any leaf under it is the same
- If the invariants hold, then:
  - Depth of any leaf ≥ black-height(root)
  - Depth of any leaf ≤ 2 x black-height(root), because every red node in the path has a black parent that is counted in black-height(root)
  - So, all root-to-leaf paths are within a factor 2 of each other in length!

2-3-4 Trees

- Red-Black Trees can be viewed as an optimised representation of a perfectly balanced non-binary search tree, with 2, 3 or 4 children per node
- Non-Binary Search Tree
  - A node with d children carries d-1 keys $k_1 \leq \ldots \leq k_{d-1}$ and for i=0 to d-1, the ith child's subtree has keys in the range $[k_i,k_{i+1})$ (using $k_0=-\infty$ and $k_d=+\infty$)
    - E.g., binary nodes (d=2) have one key k_1 with child 0 holding keys $< k_1$ and child 1 holding keys $\geq k_1$
- Perfectly balanced: All leaves (empty nodes) appear at the same level
  - #leaves between 2^h and 4^h. #keys = #leaves-1

2-3-4 Trees and Red-Black Trees

- A 2-3-4 tree can be "expanded" into a red-black tree
- Idea: A node with 3 or 4 children can be viewed as a combination of 2 or 3 binary nodes
- In the expanded binary tree, the original nodes are black and newly inserted nodes are red
- Black-height of any (black) node is the same as its height in the 2-3-4 tree
- Every red node has a black parent
- Root and leaves stay black
- How do you keep a 2-3-4 tree perfectly balanced?

